PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3
4ta. práctica (tipo b)
(Primer Semestre 2026)

Indicaciones Generales:
- Tiempo estimado: 1h 50 minutos
- Se les recuerda que, de acuerdo al reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Para el desarrollo de toda la práctica debe utilizar el sistema operativo Windows.
- Para el desarrollo de las Preguntas Nro. 1, 2 y 3 puede utilizar únicamente Notepad++ o Sublime.
  Esta prohibido el uso de VISUAL STUDIO CODE o de un entorno de desarrollo integrado como INTELLIJ.
- Está permitido el uso de apuntes de clase, diapositivas, ejercicios de clase y código fuente.
  (Debe descargarlos antes de iniciar con la solución del enunciado)
- Está permitido el uso de Internet (únicamente para consultar páginas oficiales de Microsoft y Oracle). No obstante, está prohibida toda forma de comunicación con otros estudiantes o terceros.

PARTE PRÁCTICA (20 puntos)

PUEDE UTILIZAR MATERIAL DE CONSULTA.
Antes de comenzar el laboratorio, descargue todos los proyectos, apuntes, diapositivas que utilizará.
Se considerará en la calificación el uso de buenas prácticas de programación (aquellas vistas en clase).

Como parte de un proceso de reingeniería, se ha establecido modificar la **organización estructural de las clases y componentes del software** desarrollado en el Laboratorio 02, empleando un enfoque basado en modularización y reutilización.

A partir del enunciado descrito, el estudiante deberá seleccionar una de las siguientes alternativas de implementación:

- **Pregunta 1 y 2, correspondiente al desarrollo de componentes reutilizables en Java**
- **Pregunta 3 y 4, correspondiente al desarrollo de componentes reutilizables en .NET**

Considerar que la **pregunta 5 si es obligatoria**, independientemente de que tecnología ha desarrollado (JAVA o .NET)

JAVA:

PREGUNTA 1: Ejercicio de paquetes (Sin uso de IDE) (7 puntos)

En la Fig. 01, se muestra a través de un diagrama de clases, la nueva estructura de paquetes.
Se le solicita descargar los archivos fuente que se adjuntan a este enunciado. Agregue las instrucciones **package** e **import** a todas las clases, los ámbitos correspondientes de las mismas, así como la estructura de carpetas para que el programa JAVA funcione bajo la nueva estructura de paquetes solicitada. Compruebe que es posible compilar y ejecutar bajo la nueva estructura. Está prohibido utilizar el comodín * para realizar importaciones en las clases. Para compilar, sí puede utilizar el *.

Se descontarán puntos por importaciones innecesarias.

Para un mejor entendimiento, a continuación, se detallan los paquetes y las clases que deberían contener:
- **pe.edu.pucp.assessment.question.model**: Printable, Question, SingleChoiceQuestion, MultipleChoiceQuestion.
- **pe.edu.pucp.assessment.teacher.model**: Teacher.
- **pe.edu.pucp.assessment.exam.model**: Assessment, AssessmentItem.
- **pe.edu.pucp.assessment.management.model:** QuestionManager
- **pe.edu.pucp.assessment.program.main**: Main, AssessmentViewerConsole.

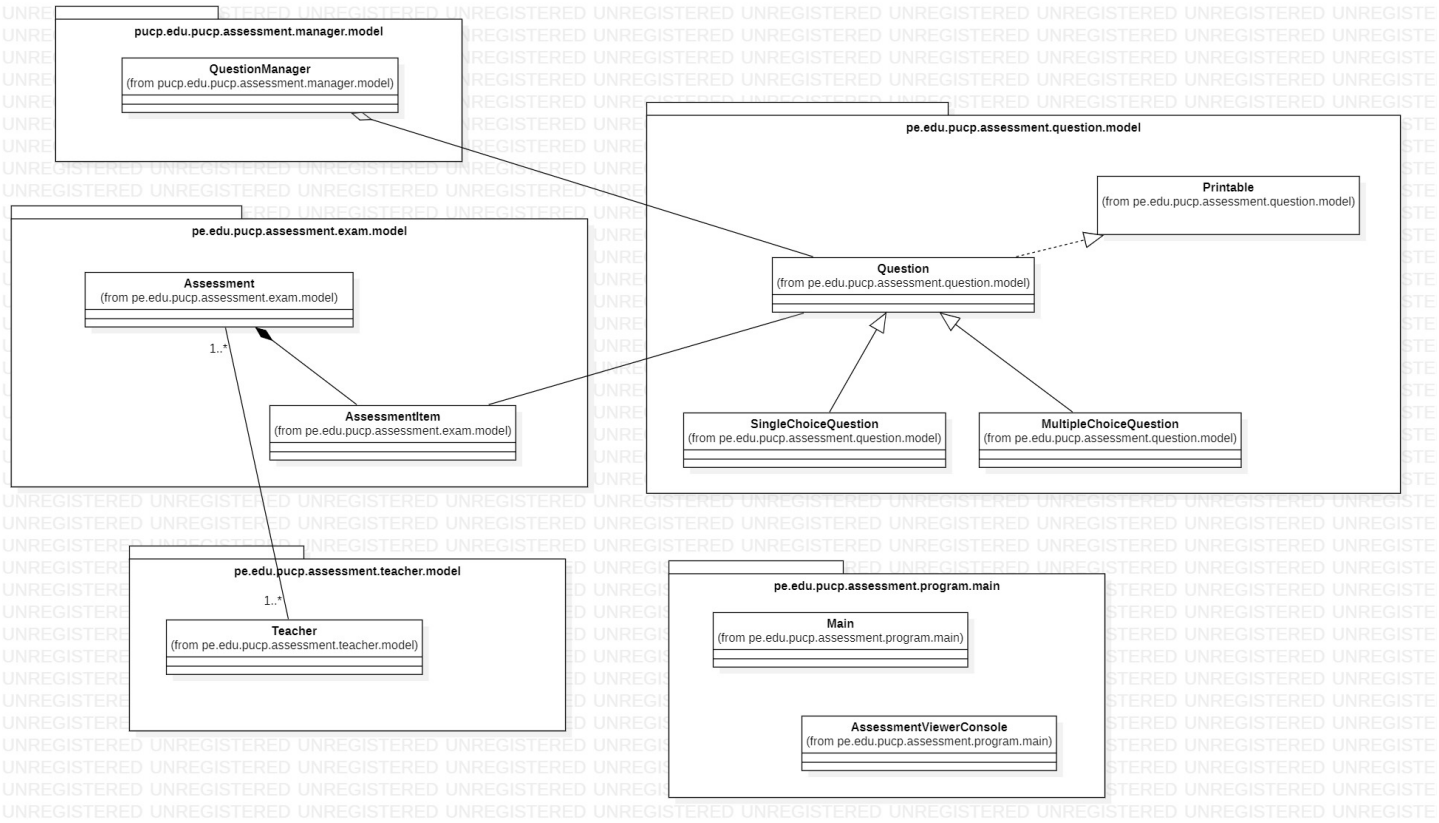

Fig. 01. Diagrama de clases – Estructura de paquetes solicitada.

Con las siguientes instrucciones puede corroborar que su programa se ha estructurado y funciona bajo el nuevo esquema:

```
javac pe/edu/pucp/assessment/question/model/*.java pe/edu/pucp/assessment/teacher/
model/*.java pe/edu/pucp/assessment/manager/model/*.java
pe/edu/pucp/assessment/exam/model/*.java pe/edu/pucp/assessment/program/main/*.java

java -cp pe/edu/pucp/assessment/question/model/*class; pe/edu/pucp/assessment/teacher/
model/*.class; pe/edu/pucp/assessment/manager/model/*.class;
pe/edu/pucp/assessment/exam/model/*.class;pe/edu/pucp/assessment/program/main/Main
```

Antes de continuar con la Pregunta 02, suba un archivo .zip a PAIDEIA que contenga TODA esta nueva estructura (con toda la estructura de carpetas y todos los paquetes que se visualizan en el diagrama).

<u>PREGUNTA 2</u>: Ejercicio de generación de librerías JAR (Sin uso de IDE) (8 puntos)
Para continuar con este ejercicio, **borre todos los archivos .class generados en el ejercicio anterior en todos los paquetes.** Realice las instrucciones en estricto orden e **indique en un archivo llamado "Pregunta02_codigoAlumno.txt"**, **las líneas de comandos utilizadas por consola** para alcanzar cada actividad:

1. Compile únicamente las clases contenidas en **pe.edu.pucp.assessment.question.model** desde el directorio raíz de trabajo.
2. Genere un componente .jar llamado "**assessmentQuestionModel.jar**" que contenga la estructura y clases en Bytecode contenidas en: **pe.edu.pucp.assessment.question.model**.
Una vez creado el componente, borre todos los archivos .java, .class, y estructura de carpetas que corresponden al componente creado " **assessmentQuestionModel.jar**" (**pe.edu.pucp.assessment.question.model**).

3. Compile únicamente las clases contenidas en **pe.edu.pucp.assessment.teacher.model** desde el directorio raíz de trabajo.
4. Genere un componente .jar llamado "**assessmentTeacherModel.jar**" que contenga la estructura y clases en Bytecode contenidas en: **pe.edu.pucp.assessment.teacher.model**.
Una vez creado el componente, borre todos los archivos .java, .class, y estructura de carpetas que corresponden al componente creado " **assessmentTeacherModel.jar**" (**pe.edu.pucp.assessment.teacher.model**).
Prosiga con las siguientes instrucciones.

5. Compile las clases contenidas en **pe.edu.pucp.assessment.manager.model** desde el directorio raíz de trabajo utilizando el .jar previamente creado: "**assessmentQuestionModel.jar**".
4. Genere un componente .jar llamado "**assessmentManagerModel.jar**" que contenga la estructura y clases en Bytecode contenidas en: **pe.edu.pucp.assessment.manager.model**.

Una vez creado el componente, borre todos los archivos .java, .class, y estructura de carpetas que corresponden al componente creado "**assessmentManagerModel.jar**" (**pe.edu.pucp.assessment.manager.model**). Prosiga con las siguientes instrucciones.

5. Compile las clases contenidas en **pe.edu.pucp.assessment.exam.model** desde el directorio raíz de trabajo utilizando los .jar previamente creados: " **assessmentTeacherModel.jar**" y "**assessmentQuestionModel.jar**".
6. Genere un componente .jar llamado "**assessmentExamModel.jar**" que contenga la estructura y clases en Bytecode contenidas en: **pe.edu.pucp.assessment.exam.model**.
Una vez creado el componente, borre todos los archivos .java, .class, y estructura de carpetas que corresponden al componente creado "**assessmentExamModel.jar**" (**pe.edu.pucp.assessment.exam.model**). Prosiga con las siguientes instrucciones.

7. Compile las clases contenidas en **pe.edu.pucp.assessment.program.main** desde el directorio raíz de trabajo utilizando los .jar previamente creados: "**assessmentQuestionModel.jar**", "**assessmentTeacherModel.jar**", "**assessmentManagerModel.jar**" y "**assessmentExamModel.jar**"
8. Genere un componente **.jar ejecutable** llamado "**assessmentPrincipal.jar**" que contenga la estructura y clases en Bytecode contenidas en **pe.edu.pucp.assessment.program.main**. Comprueba que funcione. Prosiga con la siguiente instrucción.

9. Además de las instrucciones ejecutadas, incorpore en el archivo "**Pregunta02_codigoAlumno.txt**" el contenido que debería tener el **MANIFEST** de "**assessmentPrincipal.jar**" para que pueda funcionar como componente ejecutable.

Suba a PAIDEIA el archivo de texto o documento de texto con todas las instrucciones utilizadas por consola para alcanzar las actividades solicitadas. Además, adjunte también todos los archivos JARs generados y el MANIFEST.MF utilizado para la creación de assessmentPrincipal.jar.

C# .NET

PREGUNTA 3: Ejercicio de librerías (Sin uso de IDE) (7 puntos)

En la Fig. 02, se muestra, a través de un diagrama de clases, la nueva estructura modular del sistema.

Se le solicita descargar los archivos fuente del proyecto de Consola que se adjuntan a este enunciado. Organice las clases en los namespaces correspondientes, defina adecuadamente los modificadores de acceso de las clases y miembros, y configure las referencias necesarias para que el programa en .NET funcione bajo la nueva estructura solicitada. Compruebe que es posible compilar y ejecutar correctamente la solución bajo dicha organización.

Se descontarán puntos por referencias innecesarias entre proyectos o por una organización incorrecta de los namespaces.

Para un mejor entendimiento, a continuación, se detallan los proyectos y las clases que deberían contener:

- **Assessment.QuestionModel:** IPrintable, Question, SingleChoiceQuestion, MultipleChoiceQuestion.
- **Assessment.TeacherModel:** Teacher.
- **Assessment.ExamModel:** Assessment, AssessmentItem.
- **Assessment.ManagementModel:** QuestionManager.
- **Assessment.ConsoleApp:** Program, AssessmentViewerConsole.

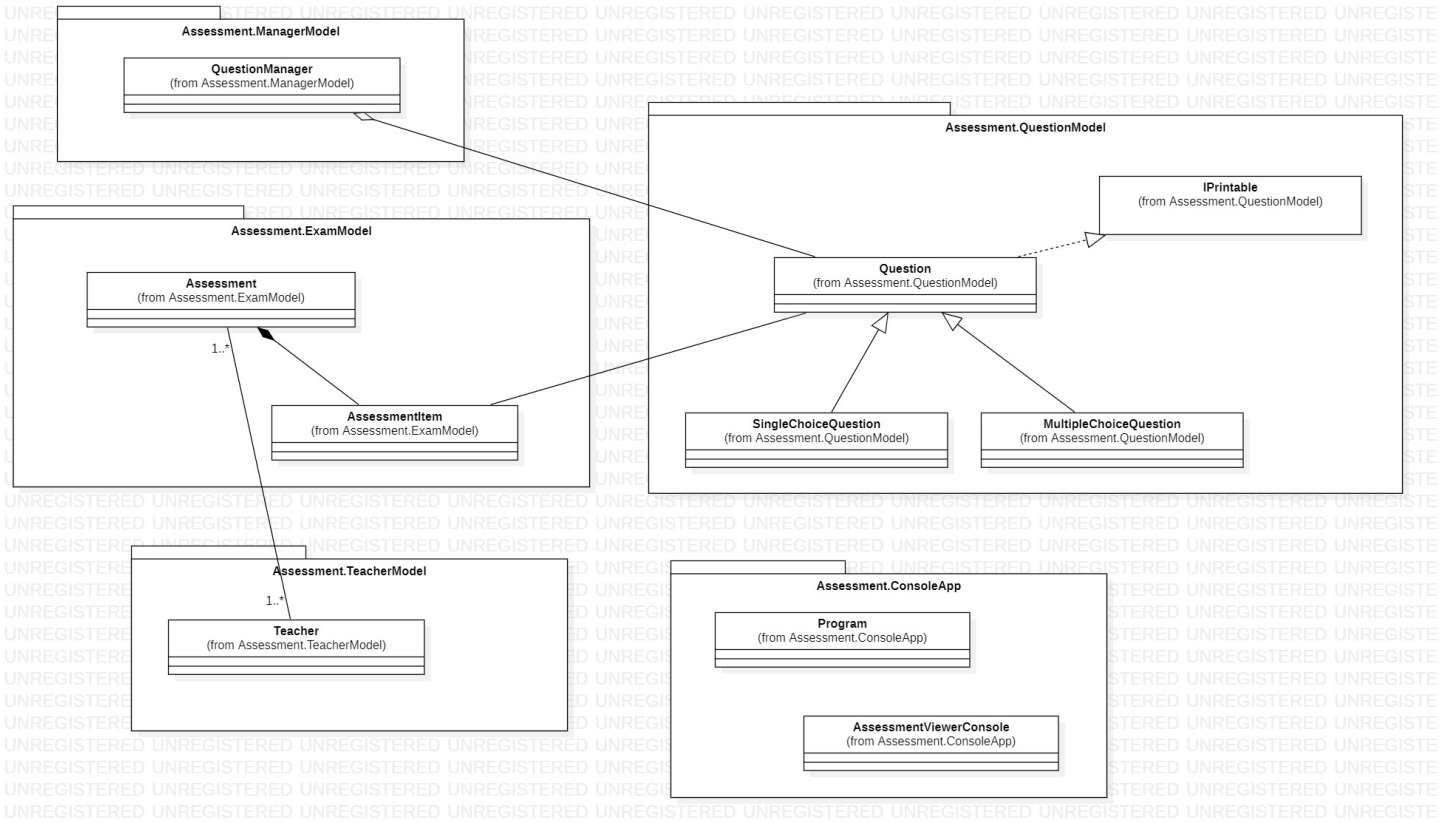

Fig. 02. Diagrama de clases – Estructura de namespace solicitada.

Con las siguientes instrucciones puede corroborar que su programa se ha estructurado y funciona bajo el nuevo esquema:

```
dotnet build

dotnet run
```

Antes de continuar con la Pregunta 04, suba un archivo .zip a PAIDEIA que contenga TODA la solución desarrollada, incluyendo la estructura completa del proyecto, carpetas y archivos fuente, conforme al esquema solicitado.

<u>PREGUNTA 4</u>: Ejercicio de generación de bibliotecas de clases DLL en .NET (Sin uso de IDE) (8 puntos)

Para continuar con este ejercicio, elimine las carpetas bin y obj generadas en el ejercicio anterior en todos los proyectos de la solución.
Realice las instrucciones en estricto orden e indique en un archivo llamado **Pregunta04_codigoAlumno.txt** las líneas de comando utilizadas por consola para alcanzar cada actividad:

1. Cree un proyecto de biblioteca de clases llamado AssessmentQuestionModel.
2. Coloque en este proyecto las clases que corresponden al namespace Assessment.QuestionModel.
3. Compile únicamente el proyecto **AssessmentQuestionModel** desde el directorio raíz de trabajo.
4. Verifique que se genere correctamente el componente **AssessmentQuestionModel.dll**, correspondiente a las clases contenidas en dicho proyecto.

Una vez generado el componente, elimine los archivos compilados y temporales correspondientes al proyecto, es decir, las carpetas bin y obj de AssessmentQuestionModel.

5. Cree un proyecto de biblioteca de clases llamado AssessmentTeacherModel.
6. Coloque en este proyecto las clases que corresponden al namespace Assessment.TeacherModel.
7. Compile únicamente el proyecto AssessmentTeacherModel desde el directorio raíz de trabajo.
8. Verifique que se genere correctamente el componente AssessmentTeacherModel.dll, correspondiente a las clases contenidas en dicho proyecto.

Una vez generado el componente, elimine los archivos compilados y temporales correspondientes al proyecto, es decir, las carpetas bin y obj de AssessmentTeacherModel.

Prosiga con las siguientes instrucciones.

9. Cree un proyecto de biblioteca de clases llamado AssessmentManagementModel.
10. Coloque en este proyecto las clases que corresponden al namespace Assessment.ManagementModel.
11. Compile el proyecto AssessmentManagementModel desde el directorio raíz de trabajo, utilizando la referencia al proyecto o componente previamente generado: Assessment.QuestionModel.
12. Verifique que se genere correctamente el componente AssessmentManagementModel.dll, correspondiente a las clases contenidas en dicho proyecto.

Una vez generado el componente, elimine los archivos compilados y temporales correspondientes al proyecto, es decir, las carpetas bin y obj de AssessmentManagementModel.

Prosiga con las siguientes instrucciones.

13. Cree un proyecto de biblioteca de clases llamado AssessmentExamModel.
14. Coloque en este proyecto las clases que corresponden al namespace Assessment.ExamModel.
15. Compile el proyecto AssessmentExamModel desde el directorio raíz de trabajo, utilizando las referencias previamente definidas a los proyectos o componentes AssessmentTeacherModel y AssessmentQuestionModel.
16. Verifique que se genere correctamente el componente AssessmentExamModel.dll, correspondiente a las clases contenidas en dicho proyecto.

Una vez generado el componente, elimine los archivos compilados y temporales correspondientes al proyecto, es decir, las carpetas bin y obj de AssessmentExamModel.

Prosiga con las siguientes instrucciones.

17. Compile el proyecto de Consola AssessmentConsoleApp desde el directorio raíz de trabajo, el cual solo debe tener las clases que pertenecen al namespace Assessment.ConsoleApp utilizando las referencias previamente definidas a los proyectos o componentes:
    - AssessmentQuestionModel
    - AssessmentTeacherModel
    - Assessment.ManagementModel
    - Assessment.ExamModel
18. Ejecute la aplicación de consola y compruebe que funcione correctamente.
19. Además de las instrucciones ejecutadas, incorpore en el archivo **Pregunta04_codigoAlumno.txt** los comandos utilizados para:
    - Crear los proyectos.
    - Agregar las referencias entre proyectos.
    - Compilar cada componente.
    - Ejecutar la aplicación principal.

Suba a PAIDEIA el archivo de texto o documento de texto con todas las instrucciones utilizadas por consola para alcanzar las actividades solicitadas. Además, adjunte también todos los archivos .dll generados y la solución completa desarrollada.

BASE DE DATOS (OBLIGATORIO)

PREGUNTA 5: Creación de un motor de base de datos MySQL en AWS Academy (5 puntos)

Se le solicita ingresar al servicio de AWS Academy (**https://www.awsacademy.com/vforcesite/LMS_Login**) y crear una instancia o motor de base de datos MySQL con los siguientes parámetros:

- Método de creación de base de datos: **Creación estándar**
- Tipo de motor: **MySQL**
- Edición: **Comunidad de MySQL**
- Versión del motor: **MySQL 8.0.36**
- Plantilla: **Capa gratuita**
- Identificador de instancias de bases de datos: **prog3-labs-1inf30**
- Nombre de usuario maestro: **admin**
- Master password: **labs1inf3020261**
- Clase de instancia de base de datos: **db.t4g.micro (2 vCPUs, 1 GiB RAM)**
- Capacidad de disco duro para la BD: **20 GiB SSD (disco sólido) sin posibilidad de escalado automático**
- Acceso público: **SI**
- Puerto de la base de datos: **3306**
- Nombre de la base de datos inicial: **assessmentlabs**
- Habilitar el cifrado: **NO**

- **Configure las reglas de entrada para que cualquier IP o computador tenga acceso a su base de datos.**
- **IMPORTANTE: El motor de base de datos debe ser configurado para que cualquier computador o IP pueda acceder al mismo. Si esta configuración no es realizada y no es posible conectarse a su motor de base de datos desde la computadora de los jefes de práctica o docente, entonces no se considerará puntaje alguno.**

Una vez creada su base de datos, se le solicita comprobar que funciona correctamente. Se le solicita dejar la base de datos **ACTIVA** en AWS y colocar los datos de conexión en un archivo llamado **datosConexion_códigoAlumno.txt** (en el archivo deberá colocar el *hostname*, el *username*, el *password* y el nombre de la BD o esquema). Compruebe que es posible acceder a su base de datos creada utilizando MySQL Workbench y **suba el archivo .txt a PAIDEIA**.

Los siguientes aspectos pueden conllevar a un descuento significativo en su propuesta de solución:

- No seguir las instrucciones establecidas en el enunciado.
- Que los programas no compilen.
- Que los programas presenten errores en tiempo de ejecución.
- Que los proyectos tengan referencias innecesarias.

15 de abril del 2026